

Performance Characterization of the Alpamayo 1.5 End-to-End Driving Model on NVIDIA Jetson AGX Thor

A latency, energy, memory, and accuracy study of a
vision-language trajectory planner deployed in Autoware

Author: _____

University of Michigan · teadem@umich.edu

Host platform: NVIDIA Jetson AGX Thor (Blackwell, SM110)
Model: Alpamayo 1.5 (10B) · Runtime: ROS 2 / Autoware

Report date: September 2026

Methodology frozen 2026-08-18; experiments extended and status refreshed 2026-09-02

Abstract

This report characterizes the on-device inference behavior of *Alpamayo 1.5*, a 10-billion-parameter vision-language driving model that consumes multi-camera imagery and ego odometry and emits a planned ego trajectory for the Autoware autonomy stack. All measurements were collected by replaying recorded driving logs through the production ROS 2 node on an NVIDIA Jetson AGX Thor and instrumenting the node for per-inference timing, Jetson rail power, analytical FLOP counts, KV-cache/GPU-memory footprint, and (for Waymo) ego trajectory accuracy against replayed odometry. Two datasets are used throughout: a four-camera **Waymo** moving segment (16 images per inference) and a one-camera **MCity** segment (4 images per inference). Under default runtime parameters the model runs at roughly 3.00 s–4.00 s per inference on Waymo and 2.00 s–2.60 s on MCity, draws a mean GPU-rail power of 20.00 W to 33.00 W, and spends the large majority of its GPU time in memory-bound autoregressive reason-token decode (71.00 % of stage time). We present controlled sweeps over the four principal runtime levers—*diffusion steps*, *trajectory samples (K)*, *numeric precision*, and *reasoning-token cap*—and show that trajectory-sample count and precision dominate the latency/energy budget, while the token cap is essentially inert because generated reasoning length rarely approaches the cap. FP16 is found to be both slower and higher-energy than the BF16 default on this Thor stack. Ego ADE/FDE against replayed odometry stays sub-metre across all runnable Waymo configurations. A controlled same-dataset camera-count ablation shows camera count is the strongest accuracy lever (1→4 cameras cuts ADE $3.6\times$ at $2.9\times$ energy), and a best-of- K analysis shows the model’s unpublished samples are 40–50% more accurate than the trajectory it actually emits. The report documents each analysis with its methodology, exact run configuration, and result plots, and closes with a status audit and prioritized list of remaining work.

Contents

1	Introduction	1
1.1	Platform and model under test	1
1.2	Datasets	1
2	Common Methodology and Metric Definitions	2
2.1	End-to-end replay harness	2
2.2	Formal metric definitions	2
3	Canonical Baselines	4
4	Latency Analysis	4
4.1	Baseline latency, throughput, and input freshness	5
4.2	Effect of the maximum generation length (reasoning-token cap)	7
4.3	Effect of the number of diffusion steps	10
4.4	Effect of quantization (numeric precision)	11
5	Latency Breakdown by Stage	13
6	Bottleneck Identification with Nsight Profiling	13
7	Power and Energy Analyses	16
8	Predicted Trajectory Accuracy	18
9	FLOP / TOPS	19
10	KV Cache Size	21
11	Trajectory-Sample (K) Sweep	23
12	Camera-Count Ablation	25
13	Token-Limit Behavior	26
14	Camera Stream Validation and Freshness	27
15	Navigation-Text Input	28
16	Status Audit and Remaining Work	29
17	Conclusion	30

1 Introduction

Alpamayo 1.5 is an end-to-end driving model: rather than the classical detect–track–predict–plan pipeline, a single vision–language model (VLM) ingests the recent camera history and ego state, produces a chain-of-thought reasoning trace, and hands a latent to a diffusion *expert* that denoises a trajectory over a small number of steps. The model is wrapped as a ROS 2 node and publishes an Autoware `Trajectory` message, chain-of-thought text, and visualization markers. The inference path is strictly serial—one GPU, one inference in flight at a time—so end-to-end latency is the sum of four stages: vision encode, VLM prefill, autoregressive reason-token decode, and trajectory-diffusion expert.

Goal of this work. The purpose of the internship characterization was to answer, quantitatively and reproducibly, *how the model behaves as a deployed real-time system on edge hardware*: how long each inference takes, how much energy and GPU memory it costs, how those costs scale with the model’s tunable runtime knobs, and whether the planned trajectory remains accurate as those knobs are turned down for speed. Every claim in this report is backed by an artifact under the workspace `results/` tree, and every number is defined by an explicit formula in Section 2.

Two operating rules. The study follows two standing rules that make the numbers comparable:

1. **Dataset rule.** Every non-ground-truth characterization is run for *both* Waymo and MCity when the data and harness exist. Ground-truth dependent accuracy (ADE/FDE, actor collision/clearance) remains Waymo-only until a trusted MCity future-trajectory target is added.
2. **Default-parameter rule.** A sweep varies *only* the parameter under test; all other model/runtime parameters stay at the current node defaults: `precision_mode=bf16`, `max_generation_length=64`, `num_trajectory_samples=1`, `num_diffusion_steps=5`. Dataset-specific I/O (camera topics, odometry topic, replay rate, QoS) is held at its dataset default.

1.1 Platform and model under test

Table 1: System under test.

Component	Configuration
Accelerator	NVIDIA Jetson AGX Thor (Blackwell class, SM110)
Model	Alpamayo 1.5, ≈ 10 B parameters, BF16 weights
Reasoning tokenizer	<code>nvidia/Cosmos-Reason2-8B</code> (for FLOP token counting)
Runtime	ROS 2 node (<code>alpamayo_node_loki</code>), single-inference worker
Expert	Native PyTorch diffusion denoiser (default)
Attention	Flash / memory-efficient attention, autocast BF16
Processed images	560×1008 , 4 temporal frames per camera
Power sensing	Jetson INA rails via <code>/sys/class/hwmon</code> , 200 Hz sampler

1.2 Datasets

The Waymo segment carries a genuine future ego-odometry timeline, which we use as a *sanity* target for ADE/FDE (Section 8)—this is not the official Waymo planning benchmark. MCity provides one compressed camera and `/gps/odom` but no trusted future-trajectory target, so all MCity accuracy figures are reported as N/A by design.

Table 2: The two replay datasets. The Waymo-vs-MCity comparison is simultaneously a camera-count comparison; see Section 14.

Dataset	Cameras	Images/inf.	Odometry	Replay rate	GT accuracy
Waymo (moving)	4	16	replayed ego odom	0.1×	ego ADE/FDE (sanity)
MCity	1	4	/gps/odom	0.5×	N/A (no trusted GT)

2 Common Methodology and Metric Definitions

2.1 End-to-end replay harness

Waymo experiments run through `scripts/run_experiment.sh`; MCity through `scripts/run_mcity_experiment.sh`. Each run records a `config.env` provenance file, starts the power sampler, waits for the node to report the model is ready, starts camera republishing, the FLOP logger, predicted-trajectory / odometry logging, and the camera validator, optionally starts the stamped navigation-text publisher, then plays the bag with `/clock` at the configured rate. After drain, helper process groups are torn down and offline post-processing computes energy and (Waymo) ego ADE/FDE. The per-run artifacts that every downstream number is derived from are listed in Table 3.

Table 3: Primary per-run artifacts produced by the harness.

Artifact	Meaning
<code>config.env</code>	Dataset, token cap, diffusion steps, replay rate, node suffix, nav settings, params file.
<code>inference_metrics.csv</code>	One row per completed inference: ROS + wall-clock stamps, stage/model durations, token counts, nav text, publication metadata.
<code>power_samples.csv</code>	Jetson INA rail power at 200 Hz, wall-clock stamped.
<code>energy_per_inference.csv</code>	Per-inference rail energy and average power, integrated over each model window.
<code>scene_flops.csv</code>	Analytical per-inference FLOP estimate by stage and effective TFLOP/s.
<code>waymo_eval.csv</code>	Per-prediction ego ADE/FDE against replayed future odometry.
<code>camera_validation.txt</code> , <code>camera_receipts.csv</code>	Camera counts, drops/duplicates, trajectory count, per-inference input freshness.
<code>kv_memory_metrics.csv</code>	Per-inference KV tensor footprint and CUDA allocator peaks.

2.2 Formal metric definitions

Latency. All node timestamps use wall-clock `time.time()` so they can be joined against power samples. The reported model duration for inference i is the model window

$$T_i = \text{end_wall_s}_i - \text{model_start_wall_s}_i, \quad (1)$$

while `duration_s` additionally includes CPU/GPU input preparation. Per-stage latency is reported only where NVTX/stage instrumentation exists for that run.

GPU energy and power. Using the GPU rail power $P_{\text{gpu}}(t) = \text{vdd_gpu_w}(t)$ over the model window $[a_i, b_i]$,

$$E_{\text{gpu},i} = \int_{a_i}^{b_i} P_{\text{gpu}}(t) dt, \quad \bar{P}_{\text{gpu},i} = \frac{E_{\text{gpu},i}}{b_i - a_i}, \quad (2)$$

computed by trapezoidal integration with linear interpolation at the window edges. Net energy subtracts an idle baseline P_{idle} estimated from samples outside all guarded inference windows:

$$E_{\text{gpu},i}^{\text{net}} = \int_{a_i}^{b_i} \max(P_{\text{gpu}}(t) - P_{\text{idle}}, 0) dt. \quad (3)$$

Peak GPU power for an inference is $\max_t P_{\text{gpu}}(t)$ inside the model window; because it is sampler-cadence dependent, the sampler frequency (200 Hz) is reported alongside every peak.

Trajectory accuracy (ADE/FDE). For prediction i with K_i matched future waypoints, predicted points $p_{i,k}$ and aligned target points $g_{i,k}$ in a common local frame,

$$\text{ADE}_i = \frac{1}{K_i} \sum_{k=1}^{K_i} \|p_{i,k} - g_{i,k}\|_2, \quad \text{FDE}_i = \|p_{i,K_i} - g_{i,K_i}\|_2. \quad (4)$$

Reported ADE/FDE are the arithmetic means of $\text{ADE}_i, \text{FDE}_i$ over predictions with enough aligned future points.

Analytical FLOP / achieved throughput. The analytical operation model (from `scripts/flop_model.py`) counts transformer matmuls with multiply-add as 2 FLOPs and excludes layernorm, softmax, RoPE, bias, and residual adds:

$$\text{decode TFLOP} = R \cdot 0.0171, \quad \text{total TFLOP} = 63.81 + \text{decode TFLOP}, \quad (5)$$

where R is the generated reasoning-token count and 63.81 TFLOP is the fixed vision+prefill+expert baseline calibrated to the Waymo 4-camera prompt ($P = 3331$ prompt tokens, 17.1 GFLOP per decode token). Achieved throughput is obtained only after joining to measured latency: $\text{TFLOP/s}_i = \text{total FLOP}_i / T_i$. The baseline constant is *not* camera-count-correct for MCity, so MCity FLOP is reported as an analytical logger result, not a normalized operation count.

KV cache and CUDA memory. KV bytes are summed directly from the model’s `past_key_values` tensors after VLM generation and before diffusion consumes them:

$$\text{KV MiB}_i = \frac{1}{2^{20}} \sum_{j \in \text{unique tensors}} \text{numel}_j \cdot \text{element_size}_j. \quad (6)$$

Whole-inference memory pressure is measured separately as the CUDA allocator peak delta, $\Delta_i = (\text{peak allocated during } i - \text{allocated at start}_i) / 2^{20}$, which includes KV growth *plus* activations, temporaries, and diffusion/expert working memory—it is not KV-only.

Throughput and freshness.

$$\text{inference FPS} = \frac{\text{completed inferences}}{\text{elapsed wall/sim time}}, \quad \text{trajectory FPS} = \frac{\text{published trajectories}}{\text{elapsed wall/sim time}}. \quad (7)$$

A lower inference count than camera-frame count is expected because the node has a warmup/history requirement and an active-inference guard that prevents overlapping model executions when model latency exceeds the trigger period.

3 Canonical Baselines

Methodology

The canonical baselines are archived clean replays that fix all runtime knobs and serve as the reference operating point for every later sweep. The Waymo baseline is the moving segment replayed with a `_loki` node suffix, 4 cameras and 16 images per inference; the MCity baseline is the one-camera harness run with `/gps/odom`. Both were captured at `NUM_DIFF_STEPS=2`; later default-parameter work uses the node default of 5 steps, so these are the *clean diff-2* baselines, not default-parameter baselines and not the full Waymo planning benchmark.

Configuration

Table 4: Baseline run configurations.

Parameter	Waymo baseline	MCity baseline
Source dir	results/exp_eval/moving_diff2	results/kv/mcity_diff2 (+ legacy)
Cameras / images	4 / 16	1 / 4
max_generation_length	64	64
num_diffusion_steps	2	2
num_trajectory_samples	1	1
Replay rate	0.1×	≈ 0.5× (backlog-safe)

Results

Table 5 summarizes the two baselines; the paired bar summary and latency/energy comparison are shown in Figures 1 and 2. The Waymo baseline completes 46 inferences at 3452.00 ms mean model duration, draws 30.80 W mean GPU power and 106.00 J GPU energy per inference, and holds mean ego ADE/FDE of 0.11 m/0.19 m. MCity, with a quarter of the image tokens, is faster (2640.00 ms) and lower-power (20.20 W, 52.50 J) but generates *more* reasoning tokens on average (13.2 vs 7.8), reflecting scene/route differences rather than a camera effect.

Table 5: Canonical baseline metrics (mean unless noted; $\pm$ is std).

Metric	Waymo (diff2)	MCity (diff2)
Completed inferences	46	133
Model duration (ms)	3452.5 $\pm$ 316.3	2640.0 $\pm$ 925.1
Reasoning tokens (mean / max)	7.85 / 20	13.24 / 53
Mean GPU power (W)	30.81 $\pm$ 1.13	20.23 $\pm$ 1.26
GPU energy / inf. (J)	106.07 $\pm$ 5.94	52.52 $\pm$ 14.82
Total board energy / inf. (J)	261.0 $\pm$ 18.6	169.1 $\pm$ 54.3
Total analytical TFLOP	63.94	≈ 64 (Waymo-calibrated)
Ego ADE / FDE mean (m)	0.107 / 0.194	N/A
Ego ADE / FDE median (m)	0.031 / 0.074	N/A

4 Latency Analysis

This section characterizes how end-to-end inference latency—and the GPU energy that tracks it almost linearly—responds to the model’s principal runtime knobs. It first fixes the baseline latency, throughput, and input freshness, then sweeps each latency-relevant parameter in turn: the reasoning-token cap, the number of diffusion steps, and numeric precision, holding all other

Dataset	Run	N	Images/inf	Model ms	GPU J/inf	GPU W	Tokens	ADE m	FDE m
Waymo	exp_val/moving_c	46	16.00	3452.540	106.0688	30.8139	7.848	0.107082	0.194412
MCity	mcity_top_level	133	4.00	2640.006	52.5240	20.2253	13.241	N/A	N/A

Figure 1: Cross-dataset baseline summary table (rendered from `characterization_*.csv`).

parameters at their node defaults. The trajectory-sample count K —a fourth latency lever—is treated in its own section (Section 11) because its primary payload is the best-of- K accuracy analysis.

4.1 Baseline latency, throughput, and input freshness

Methodology

Latency is taken from the wall-clock model window T_i in `inference_metrics.csv` and compared frame-to-frame, dataset-to-dataset, and across matched sweep groups. Sustained throughput is completed inferences (and published trajectories) over the replay span; because replay is slowed ($0.1\times/0.5\times$) both a *wall* FPS and a *sim* FPS are reported. Input freshness joins `camera_receipts.csv` to `inference_metrics.csv` by inference index and reports `odom_minus_camera_stamp = ros_stamp_s - last_selected_stamp_s` (age of the selected camera history) and `camera_staleness = last_received_stamp_s - last_selected_stamp_s` (whether newer frames existed but were not selected).

Configuration

Table 6: Latency / throughput / freshness measurement configuration.

Parameter	Waymo	MCity
Cameras / images per inf.	4 / 16	1 / 4
<code>max_generation_length</code>	64	64
<code>num_diffusion_steps</code>	2	2
<code>num_trajectory_samples</code>	1	1
<code>precision_mode</code>	bf16	bf16
Replay rate	$0.1\times$	$0.5\times$
Trigger period	0.10 s	0.10 s
Power sampler	200.00 Hz	200.00 Hz
Swept parameter	none (baseline operating point)	

Results

Table 7 gives sustained throughput at the baseline operating point. Because model latency (~ 3.50 s Waymo) exceeds the 0.1 s trigger period even after the $0.1\times$ slowdown, the node’s active-inference guard drops the completed-inference count well below the camera frame count—this is

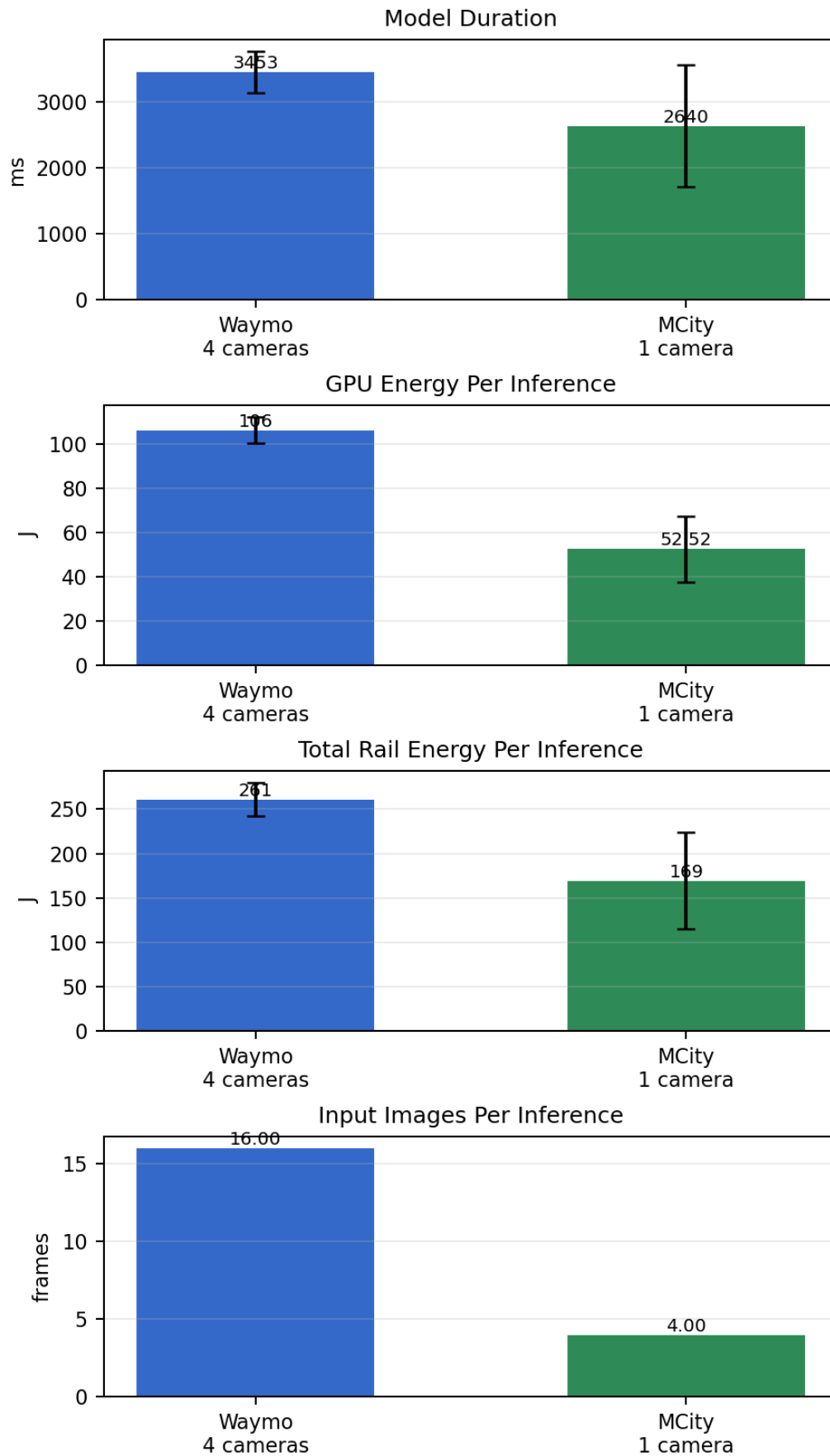


Figure 2: Waymo vs MCity mean model latency and per-inference GPU energy. MCity’s smaller prompt lowers both latency and energy despite longer reasoning traces.

expected serial-inference behavior, not frame loss. Figure 3 shows the paired throughput/token bars, and Figure 4 relates per-inference latency to scene complexity (reasoning-token count), the only strongly latency-correlated per-frame quantity.

Table 7: Sustained throughput at the baseline operating point.

Run	Completed inf.	Sim inference FPS	Wall inference FPS
Waymo baseline (exp_eval/moving_diff2)	46	2.49	0.25
MCity baseline (kv/mcity_diff2)	137	0.53	0.26

Input freshness. Table 8 reports the two per-inference freshness quantities of Section 2.2, aggregated over the baseline reruns. The selected camera history is effectively synchronous with the trajectory/odometry stamp: `odom_minus_camera_stamp` is exactly 0.00s on Waymo (camera and odometry share replay stamps) and within -64.00 ms on MCity (camera marginally newer, with one -2.3 s outlier). Crucially, `camera_staleness` = 0 everywhere—the node always selected the newest available frames, so no usable camera data was left unconsumed. The selection buffer never starved (minimum 4 selected frames, buffer depth up to 12).

Table 8: Input freshness at the baseline operating point (per-inference means).

Quantity	Waymo	MCity
Inferences aggregated	43	137
<code>odom_minus_camera_stamp</code> mean (s)	0.000	-0.064
<code>odom_minus_camera_stamp</code> min / max (s)	0.000 / 0.000	-2.296 / 0.019
<code>camera_staleness</code> mean / max (s)	0.000 / 0.000	0.000 / 0.000
Selected frames / inf. (min)	4	4
Buffer depth (max)	12	12

4.2 Effect of the maximum generation length (reasoning-token cap)

Methodology

The token-cap sweep varies `MAX_GEN_LEN` while holding replay segment, diffusion steps, camera setup, node code, and replay rate fixed, using the clean-only Waymo artifacts. Both the configured cap and the *actual* generated reasoning length are reported; a flat trend is expected if actual tokens do not approach the cap.

Configuration

Table 9: Reasoning-token-cap sweep configuration (clean-only Waymo).

Parameter	Value
Swept: <code>max_generation_length</code>	{16, 32, 64}
<code>num_diffusion_steps</code>	5
<code>num_trajectory_samples</code>	1
<code>precision_mode</code>	bf16
Cameras / images per inf.	4 / 16 (Waymo)
Replay rate	0.1×
Accuracy target	ego-odometry sanity ADE/FDE only

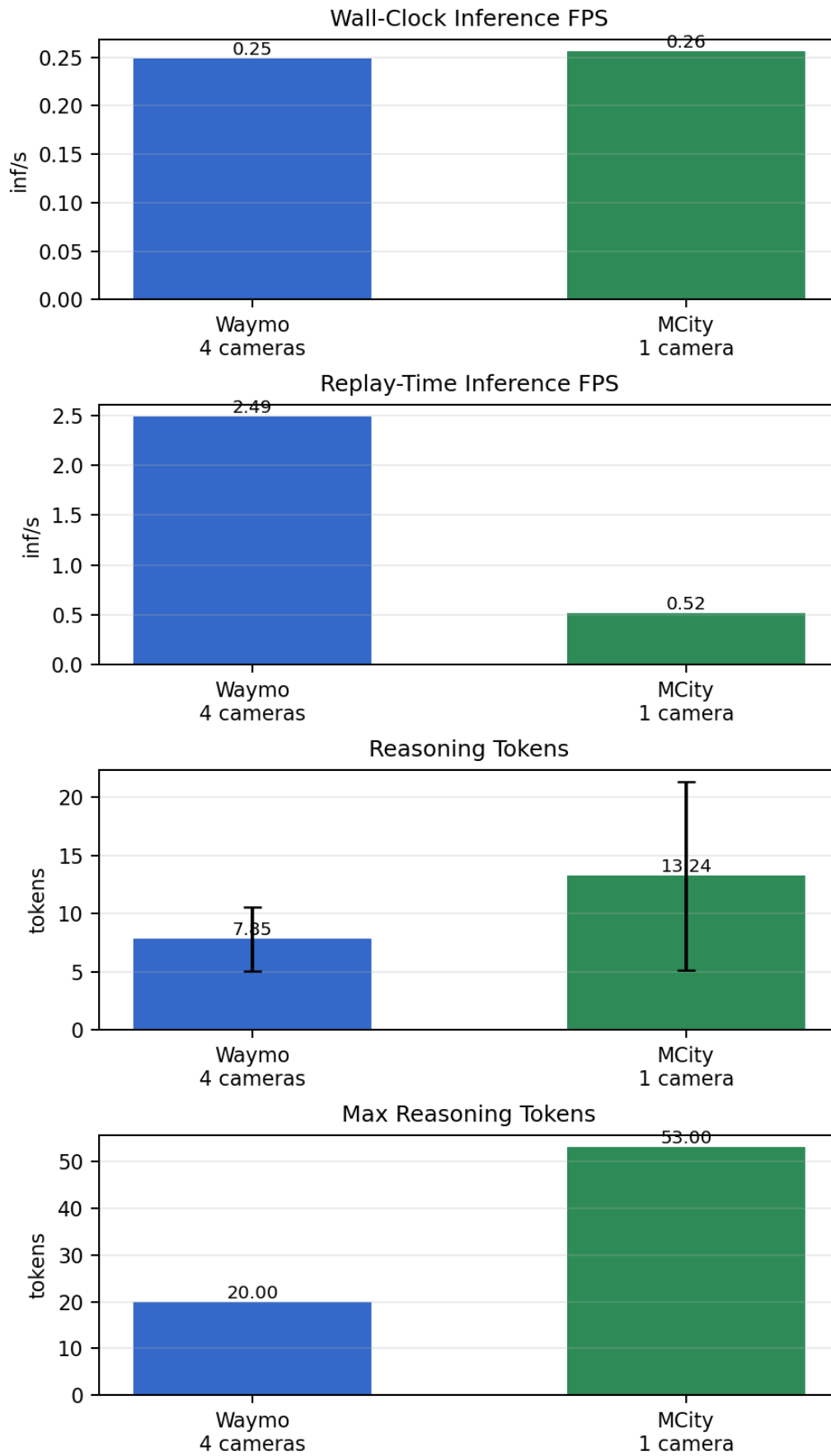


Figure 3: Paired throughput and reasoning-token bars for Waymo and MCity.

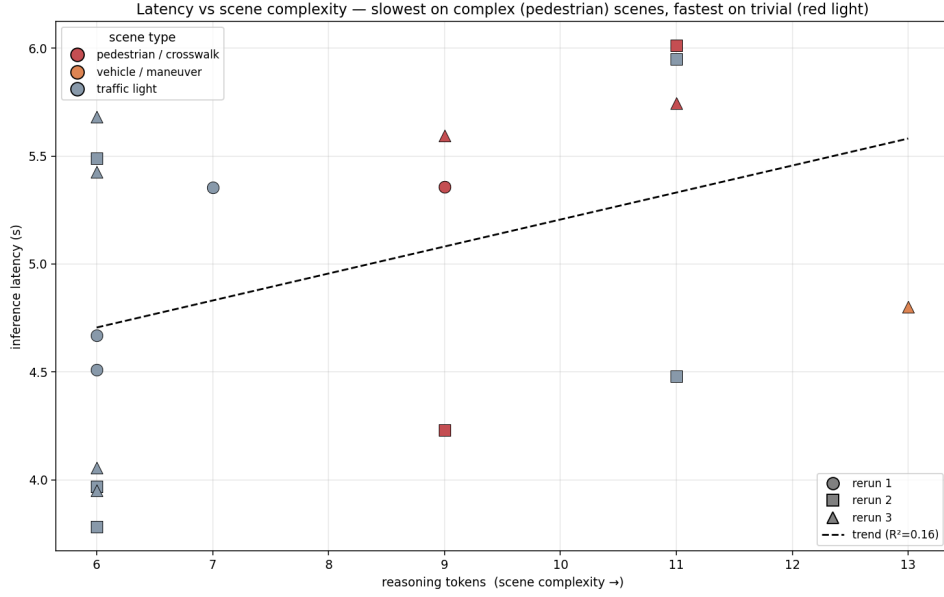


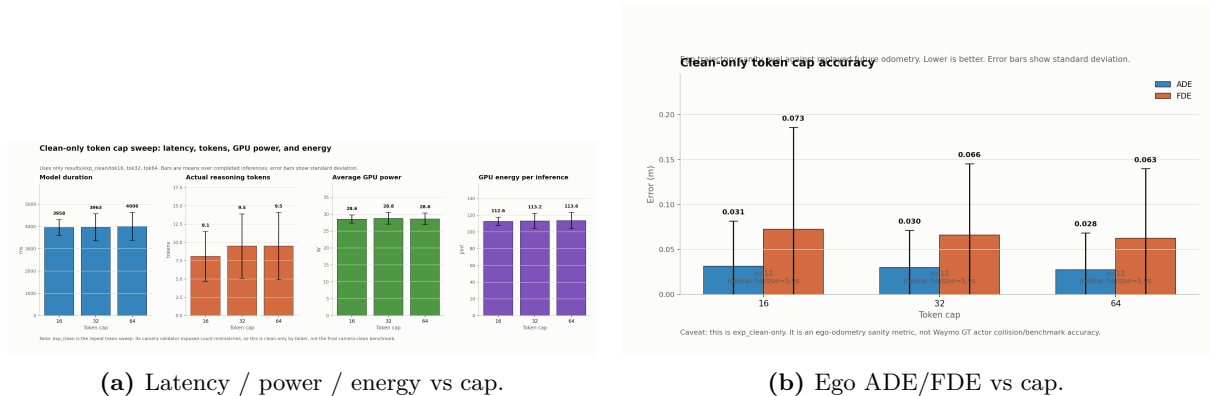
Figure 4: Per-inference model latency vs scene complexity (reasoning-token count). Decode length is the dominant per-frame latency driver, consistent with the stage profile in Section 5.

Results

The cap is effectively inert: mean generated reasoning length is 8.1/9.5/9.5 tokens against caps of 16/32/64, never hitting the cap (max-token hit rate 0% across all runs). Consequently latency (3958/3963/4007ms) and GPU energy (112.6/113.2/113.6J) are flat within noise (Table 10). ADE/FDE drift slightly *downward* with a larger cap (0.031 $\rightarrow$ 0.028m ADE), but the change is inside run-to-run variance. This confirms that on these segments the reasoning-token cap is not a useful latency lever—diffusion steps and K are. Figures 5a, 5b, and 6 show the sweep.

Table 10: Clean-only Waymo token-cap sweep (mean).

Cap	n	Actual tokens	Latency (ms)	GPU E (J)	ADE (m)	FDE (m)
16	12	8.1	3957.5	112.6	0.0314	0.0726
32	12	9.5	3962.5	113.2	0.0302	0.0664
64	12	9.5	4006.5	113.6	0.0275	0.0627



(a) Latency / power / energy vs cap.

(b) Ego ADE/FDE vs cap.

Figure 5: Token-cap sweep: cost and accuracy are both flat because generated length never reaches the cap.

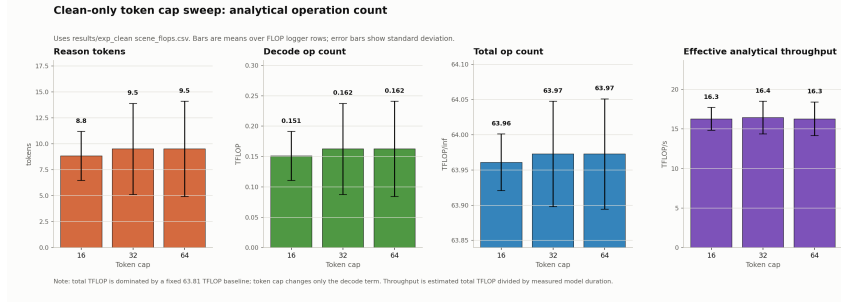


Figure 6: Analytical TFLOP and achieved TFLOP/s across token caps (≈ 16.00 TFLOP/s, memory-bound).

Dataset-matched MCity token-cap sweep

The clean token-cap sweep above was Waymo-only; the dataset rule asks for MCity too. We ran the matched MCity sweep ($\text{MAX_GEN_LEN} \in \{16, 32, 64\}$, $\text{NUM_DIFF_STEPS}=5$, $K=1$, bf16 , rate $0.5\times$; `results/token_cap_mcity/`). The result differs qualitatively from Waymo: mean reasoning length *rises* with the cap ($7.5 \rightarrow 10.3 \rightarrow 13.4$ tokens, Table 11), because MCity’s natural reasoning distribution extends past 16/32 tokens, so the lower caps genuinely truncate it (the maximum observed length sits right at the cap: 15, 25, 46). The cap is therefore a *mild active constraint* on MCity, unlike the inert Waymo case. It still does not produce a clean latency/energy trend—decode is a small share of MCity latency, and each cap completes a different subset of inferences over the replay (138/136/176), so scene mix dominates the modest differences. Note the analytical TFLOP column reports the live-logger value (≈ 64), which for MCity is the Waymo-calibrated over-count; the camera-normalized MCity baseline is 22.65 TFLOP (Section 9). Figure 7 shows the sweep.

Table 11: Dataset-matched MCity token-cap sweep (mean). Accuracy is N/A (no MCity GT).

Cap	n	Tokens (mean / max)	Latency (ms)	GPU W	GPU E (J)
16	138	7.54 / 15	2286.8	18.55	42.30
32	136	10.33 / 25	2485.5	19.09	46.77
64	176	13.40 / 46	2148.2	22.00	46.07

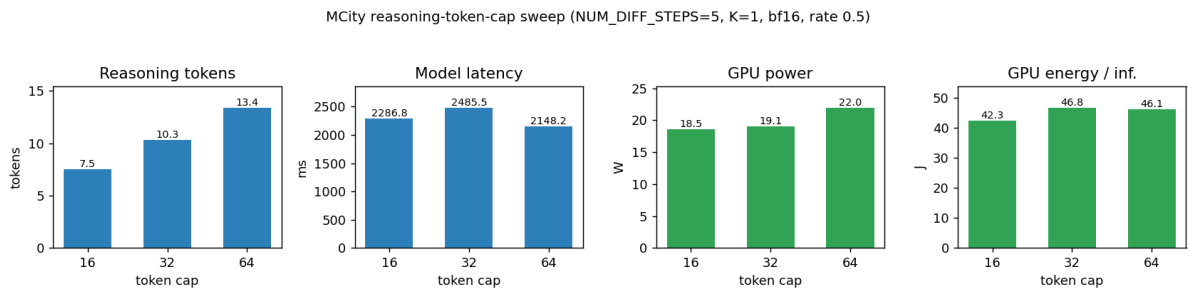


Figure 7: MCity token-cap sweep: reasoning tokens, latency, GPU power, and GPU energy. Reasoning length responds to the cap (unlike Waymo), but the cost metrics are dominated by per-cap scene mix.

4.3 Effect of the number of diffusion steps

Methodology

The diffusion-step sweep varies NUM_DIFF_STEPS while holding token cap, trajectory samples, precision, replay segment, camera setup, node code, and replay rate fixed *within* each dataset.

Because diffusion changes expert-loop work directly, it is expected to separate latency/energy more cleanly than the token cap when reasoning length stays low.

Configuration

Table 12: Diffusion-step sweep configuration.

Parameter	Waymo	MCity
Swept: num_diffusion_steps	{2, 5, 10}	
max_generation_length	64	64
num_trajectory_samples	1	1
precision_mode	bf16	bf16
Cameras / images per inf.	4 / 16	1 / 4
Replay rate	0.1×	0.5×

Results

Table 13 shows monotonic MCity growth in latency and energy with steps (2005.00 ms→2643.00 ms; 47.00 J→57.00 J). On Waymo the trend is present but partly masked by decode-length variance, since the fixed 4-camera prefill/decode dominates the step-scaled expert work. Waymo ego ADE/FDE is essentially flat across steps (ADE 0.104–0.117 m), so more diffusion steps buy little trajectory accuracy at this operating point. Figure 8 shows the full sweep.

Table 13: Diffusion-step sweep (mean). MCity accuracy is N/A by design.

Dataset	Steps	n	Latency (ms)	GPU E (J)	ADE (m)	FDE (m)
Waymo	2	45	3551.9	107.6	0.109	0.205
	5	46	3498.3	112.1	0.117	0.188
	10	43	3663.0	118.6	0.104	0.201
MCity	2	200	2005.1	47.0	N/A	N/A
	5	156	2260.1	50.2	N/A	N/A
	10	131	2642.7	56.9	N/A	N/A

4.4 Effect of quantization (numeric precision)

Methodology

The precision study runs a full replay for each viable precision mode, varying *only* precision_mode and comparing latency, GPU power/energy, KV/CUDA memory, and (Waymo) accuracy. FP8/INT8 are not run end-to-end because no end-to-end Alpamayo FP8/INT8 runtime exists in this repo; the preserved torchao INT8/FP8 VLM attempt is a negative micro-benchmark (it falls back to a slow dequant-to-BF16 path) and is explicitly excluded from replay metrics.

Configuration

Results

The headline result is counter-intuitive but robust: **FP16 is both slower and higher-energy than the BF16 default on this Thor stack**, and roughly doubles KV and CUDA-peak memory (Table 15). Waymo FP16 runs 7531.00 ms vs BF16’s 4010.00 ms and burns 221.80 J vs 116.60 J GPU per inference; MCity shows the same $\approx 2.7\times$ latency and $3\times$ energy penalty.

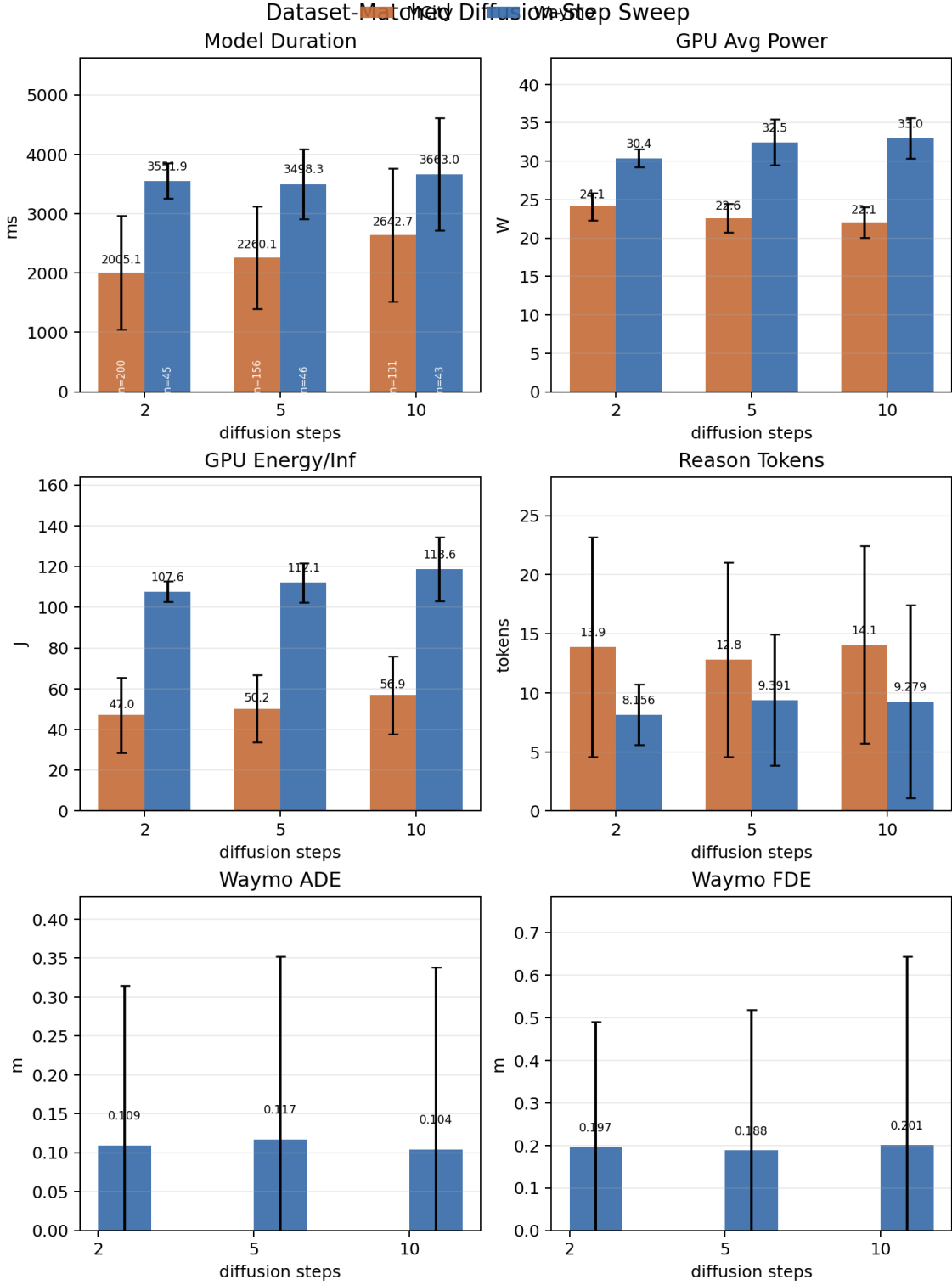


Figure 8: Dataset-matched diffusion-step sweep: latency, GPU energy/power, reasoning tokens, and (Waymo) ADE/FDE vs NUM_DIFF_STEPS.

Table 14: Precision-replay sweep configuration.

Parameter	Waymo	MCity
Swept: <code>precision_mode</code>	bf16, fp16	(fp8/int8 unsupported)
<code>num_diffusion_steps</code>	5	5
<code>max_generation_length</code>	64	64
<code>num_trajectory_samples</code>	1	1
QUANT	none	none
Cameras / images per inf.	4 / 16	1 / 4
Replay rate	0.1×	0.5×

Trajectory accuracy is statistically unchanged (Waymo ADE 0.128 vs 0.130 m), so FP16 offers no accuracy upside to justify its cost. FP8/INT8 remain unsupported for full replay. Figure 9 shows the comparison bars.

Table 15: Default-parameter precision replay (mean). FP16 is uniformly worse than BF16 here.

Dataset	Mode	n	Lat. (ms)	GPU E (J)	KV (MiB)	CUDA Δ (MiB)	ADE (m)
Waymo	BF16	41	4009.5	116.6	433.7	972.5	0.128
	FP16	23	7530.9	221.8	868.3	1523.7	0.130
MCity	BF16	163	2334.0	52.0	119.1	268.2	N/A
	FP16	73	6211.7	161.7	237.8	478.2	N/A
FP8 / INT8		0	unsupported end-to-end runtime (see *_fp8/UNSUPPORTED.md)				

5 Latency Breakdown by Stage

Methodology

End-to-end latency is the serial sum of four GPU stages—vision encode, VLM prefill, autoregressive reason decode, and the trajectory-diffusion expert. To attribute latency to each, the node installs NVTX stage ranges (`profile_stages:=true`) and an `nsys` capture is sliced per stage with `nvtx_gpu_proj_sum`. Kernel-name aggregation alone cannot separate stages that share GEMM kernels, so the measured split relies on the NVTX ranges. Captures are the single-camera MCity/staged configuration; per-stage *energy* is not available unless power samples are aligned to the same stage timestamps (listed as remaining work).

Results

The stage profile (Table 16) explains every latency observation in this report: **autoregressive reason decode is 71.40% of GPU time**, which is why decode length is the dominant per-frame latency driver (Section 4). Prefill is 12.80%, the trajectory-diffusion expert 10.20% (the step-scaled work that the diffusion and K sweeps move), and vision encode only 5.60%. Figure 10 shows the per-frame stage time series.

6 Bottleneck Identification with Nsight Profiling

Methodology

Beyond the stage shares, an Nsight Systems capture with CUDA + NVTX + OSRT traces and CUDA memory-usage tracking is used to inspect the dominant kernels inside each stage and classify whether each stage is compute- or memory-bound (Table 17). This is what turns

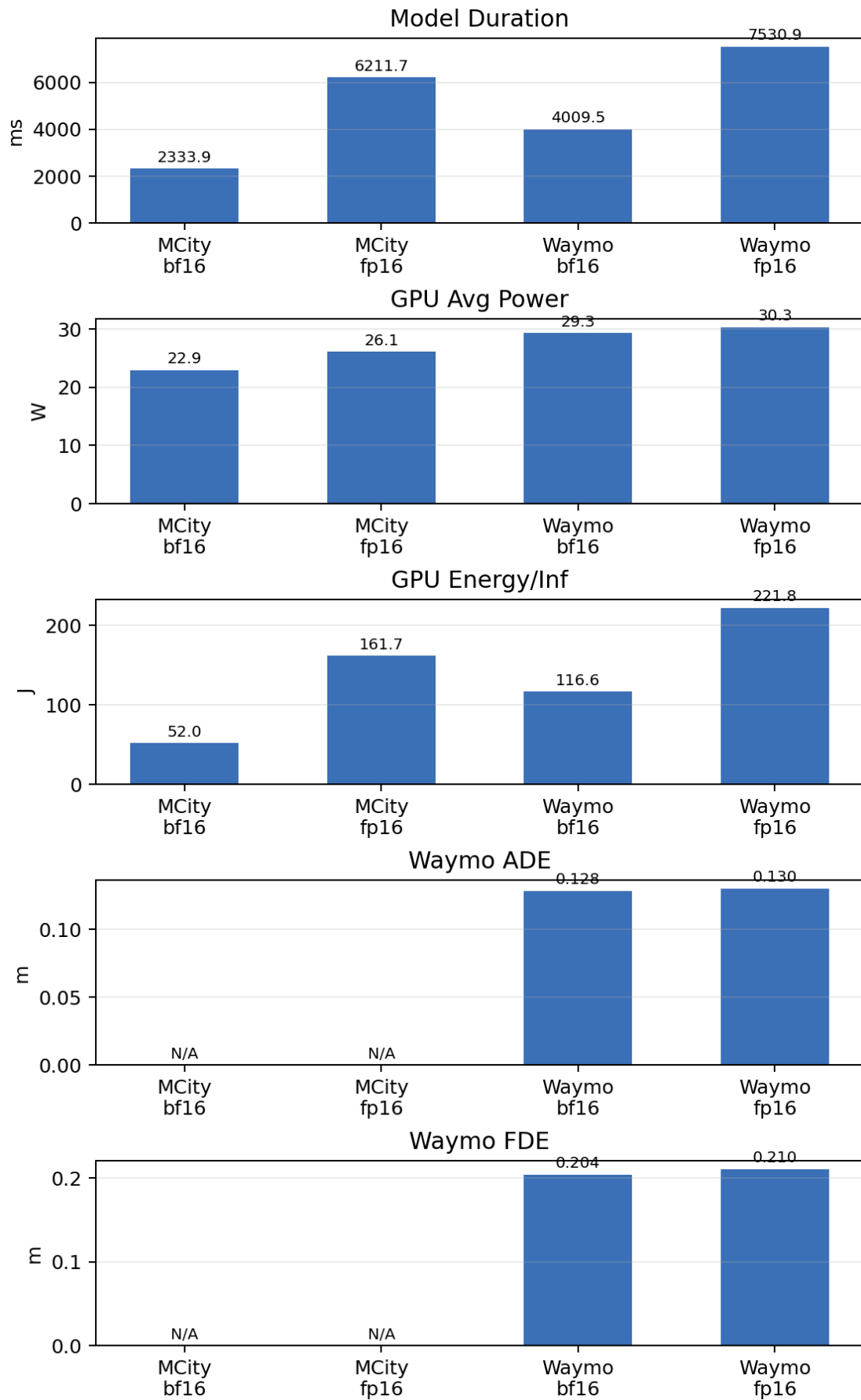
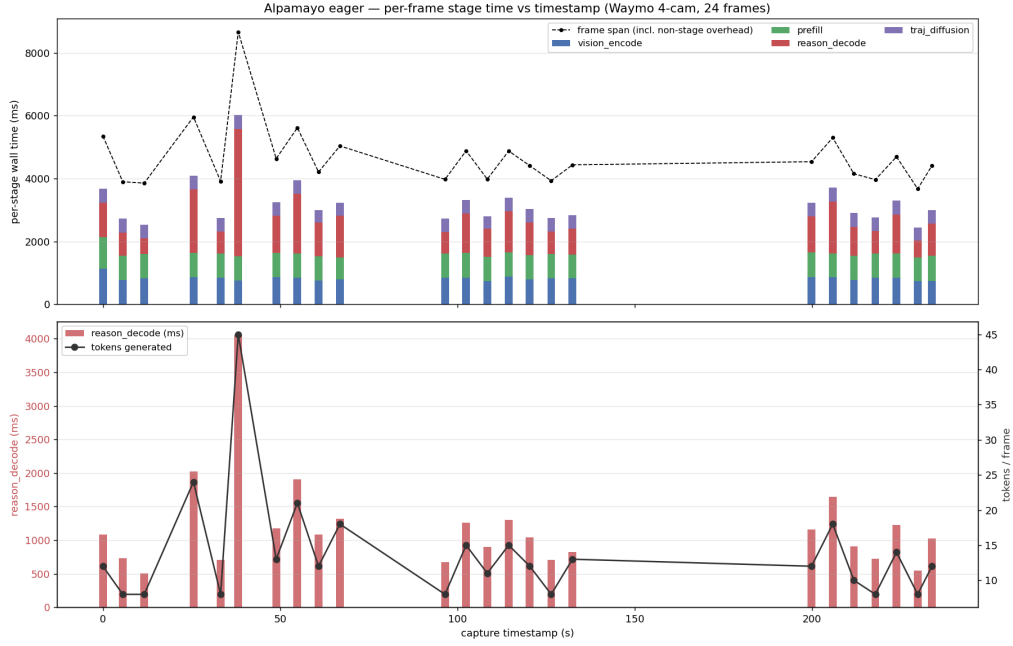


Figure 9: BF16 vs FP16 latency, GPU power, GPU energy, and (Waymo) accuracy.

Table 16: GPU-time stage breakdown (NVTX, single-camera staged capture).

Stage	GPU time (s)	Share	Character
reason_decode	55.16	71.4%	Memory-bound per-token decoder GEMMs; the latency driver.
prefill	9.91	12.8%	Compute-dense full-sequence GEMMs + flash attention.
traj_diffusion_expert	7.86	10.2%	Expert 2-CTA cluster GEMMs; scales with steps and K.
vision_encode	4.34	5.6%	ViT flash attention + biased GEMMs + heavy pointwise.

**Figure 10:** Per-frame stage timing series from the staged NVTX capture.

the “where does time go” split of Section 5 into a “what is the bottleneck” answer that guides optimization.

Configuration

Table 17: Nsight profiling configuration.

Parameter	Value
Profiler	Nsight Systems (<code>nsys</code>)
Trace domains	<code>cuda, nvtx, osrt</code>
Stage ranges	<code>NVTX, profile_stages:=true</code>
Aggregation	<code>nvtx_gpu_proj_sum</code> (measured stage split)
Capture	single-camera MCity / staged
Memory tracking	<code>-cuda-memory-usage=true</code>

Results

The bottleneck is unambiguous: the 71.00 %-of-time reason-decode stage is **memory-bandwidth-bound**, dominated by small-tile $M=1$ **nvjet** GEMMs that stream the decoder’s weights once per generated token (Table 18). This is exactly why the model sustains only ≈ 16.00 TFLOP/s despite ≈ 64.00 TFLOP of analytical work (Section 9)—it is starved on weight bandwidth, not compute. By contrast, prefill is **compute-dense** (a few large full-sequence GEMMs plus full flash attention), and the diffusion expert is a thin block of 2-CTA cluster GEMMs plus memory-efficient attention—a fusion candidate. The optimization implication is direct: latency is won by *shrinking the memory-bound decode* (fewer reasoning tokens, quantized weights, smaller KV) rather than by adding FLOPs, and by fusing the expert loop—not by touching the already-cheap vision and prefill stages.

Table 18: Dominant kernels and boundness per stage (staged **nsys** capture).

Stage	Dominant kernels	Boundness
reason_decode	small-tile $M=1$ nvjet decoder QKV/MLP GEMMs (per token), KV concat	memory-bandwidth
prefill	few large full-sequence nvjet GEMMs + full flash attention	compute
traj_diffusion_expert	2-CTA cluster nvjet GEMMs + fmha_cutlass mem-efficient atten- tion	mixed (fusion candidate)
vision_encode	ViT flash attention + biased GEMMs + heavy pointwise	mixed

7 Power and Energy Analyses

Methodology

`power_sampler.py` reads Jetson AGX Thor INA sensors under `/sys/class/hwmon` at 200 Hz, resolving rails by sensor name. The key rails are `total_w` (VDD_IN board input), `vdd_gpu_w` (GPU rail, used for GPU-only reporting), `vdd_cpu_soc_mss_w`, and `vin_sys_5v0_w`. `energy_per_inference.py` joins power to inferences by wall-clock time and integrates each rail over $[a_i, b_i]$; GPU joules per inference use `vdd_gpu_w_energy_j`, and net energy subtracts the idle baseline over the active duration. Energy is reported per inference, not per stage (stage-level energy needs NVTX-aligned power integration and is listed as remaining work).

Configuration

Table 19: GPU power / energy integration configuration.

Parameter	Value
Reported GPU rail	<code>vdd_gpu_w</code>
Integration window	model window [<code>model_start_wall_s</code> , <code>end_wall_s</code>]
Integration method	trapezoidal, linear interpolation at window edges
Idle baseline	median rail power outside guarded windows
Net energy	$\int \max(P_{\text{gpu}} - P_{\text{idle}}, 0) dt$
Power sampler rate	200.00 Hz
Datasets	Waymo ($0.1\times$) and MCity ($0.5\times$), baseline + sweeps

Results

GPU-rail energy tracks latency almost linearly across configurations because mean GPU power is nearly constant within a dataset (20.00 W to 33.00 W). At the baseline, Waymo draws 106.00 J GPU / inference vs MCity’s 52.50 J. Figure 11a decomposes MCity energy by rail, and Figure 11b shows the GPU power time series with the integrated per-inference windows.

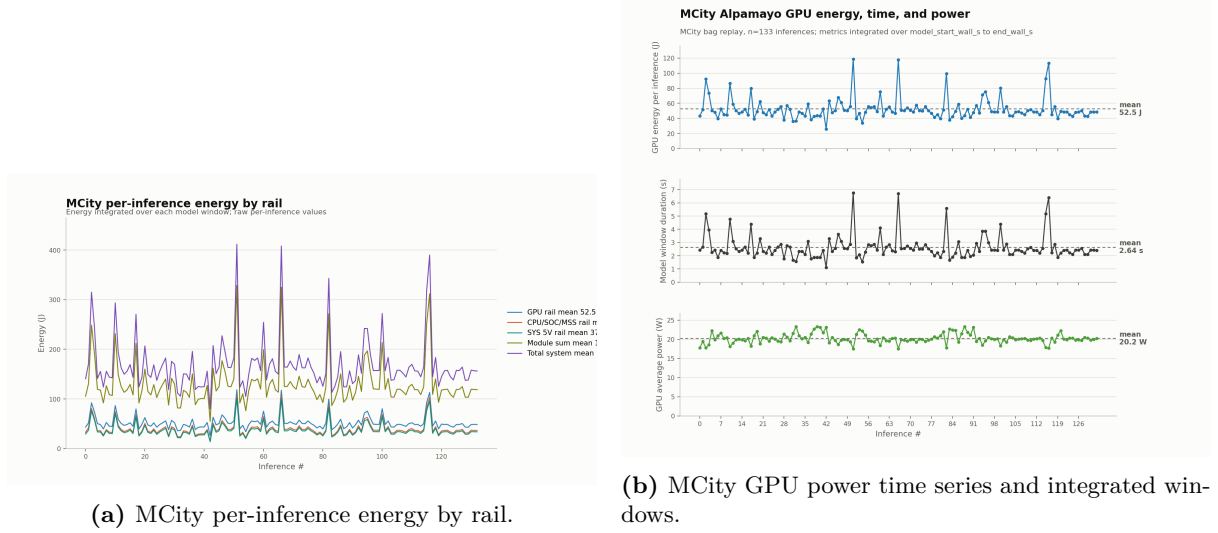


Figure 11: MCity power/energy decomposition. GPU is a minority of total board power; CPU/SOC and 5 V system rails contribute substantially to board energy.

Peak GPU power

For each inference we also take the maximum `vdd_gpu_w` sample inside the model window and summarize it across inferences (mean / p95 / peak). Peaks are sampler-cadence dependent, so the 200.00 Hz sampler frequency is reported and peaks from logs with differing cadence are never compared (Table 20).

Table 20: Peak GPU power configuration.

Parameter	Value
Reported GPU rail	<code>vdd_gpu_w</code>
Per-inference statistic	$\max_t P_{\text{gpu}}(t)$ inside the model window
Aggregation	mean / p95 / peak across inferences
Runs	diffusion-matched (Waymo + MCity, steps {2, 5, 10})
Power sampler rate	200.00 Hz

Active peak GPU power is bounded well below the module’s total board peak: on Waymo it reaches ≈ 52.00 W (e.g. `waymo_diff10` active peak 52.30 W) and on MCity ≈ 40.00 W (`mcity_diff2` 40.10 W), against total-board peaks of 123.00 W to 125.00 W. The GPU rail is therefore *not* the board-power bottleneck; CPU/SOC and system rails dominate total draw. Figure 12 shows the paired peak-GPU bars.

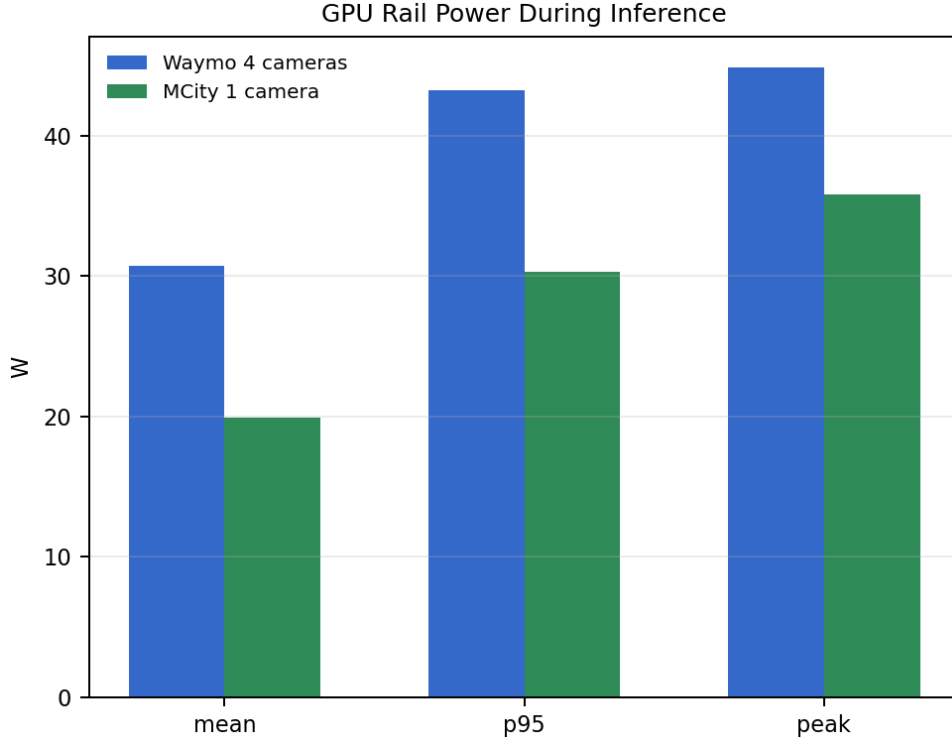


Figure 12: Peak GPU-rail power, Waymo vs MCity (200 Hz sampler).

8 Predicted Trajectory Accuracy

Methodology

Two independent accuracy views are used, both Waymo-only. **(a) Ego ADE/FDE sanity.** `evaluate_waymo_predictions.py` groups predicted trajectory points by prediction, builds a de-duplicated odometry timeline, sets each prediction’s evaluation start to `prediction_stamp + stamp_offset_s`, interpolates odometry at each `time_from_start`, transforms future poses into the evaluation-start local frame, and computes the 2D displacement errors of Section 2.2. This is an ego-odometry sanity check, not the official Waymo planning benchmark. **(b) GT actor interaction.** `evaluate_waymo_gt_actors.py` loads Waymo GT actor boxes, represents the planned ego as a 4.8×2.1 m oriented rectangle, and uses separating-axis polygon intersection for collisions and polygon-edge distance for clearance—reporting collisions, near-misses (< 1.0 m), and minimum clearance. It does *not* compute ADE/FDE.

Configuration

Table 21: Trajectory-accuracy evaluation configuration (Waymo baseline).

Parameter	Value
Dataset / operating point	Waymo baseline, <code>num_diffusion_steps=2</code>
Ego-odometry target	replayed future odometry (sanity, not GT benchmark)
Planned-ego footprint	oriented rectangle 4.8×2.1 m
GT actor classes	vehicle, pedestrian, cyclist
Actor <code>max_range_m</code>	120
<code>gt_tolerance_s</code>	0.06
<code>near_miss_m</code>	1.0
<code>actor_prefilter_m</code>	80.0

Results

Over 46 baseline predictions the model holds mean ego ADE/FDE of 0.11 m/0.19 m (median 0.03 m/0.07 m) against replayed odometry—sub-metre and stable, as expected for a short-horizon sanity target. The GT actor check reports **zero collisions** across the segment, with the nearest annotated actor a vehicle at ~ 0.68 m minimum clearance. Figure 13 shows the per-prediction error and summary bars.

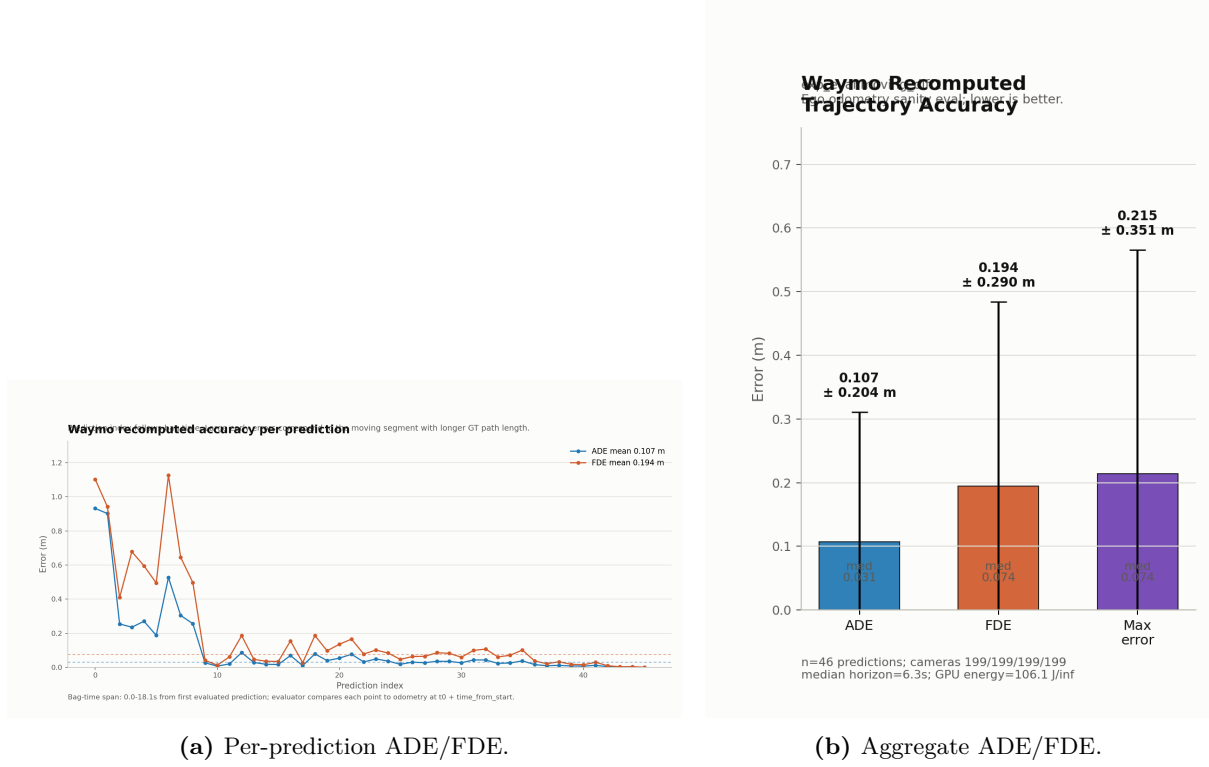


Figure 13: Waymo ego trajectory accuracy against replayed odometry (baseline diff-2 operating point).

9 FLOP / TOPS

Methodology

The FLOP logger subscribes to `/alpamayo/reasoning_stamped`, tokenizes the reasoning text with `nvidia/Cosmos-Reason2-8B`, and applies the analytical model of Section 2.2: a fixed 63.81 TFLOP vision+prefill+expert baseline (calibrated to the Waymo 4-camera, $P = 3331$ -token prompt) plus 17.10 GFLOP per decode token. Achieved TFLOP/s is total $FLOP/T_i$. Absolute values are analytical estimates suitable for *relative/stage* analysis, not hardware-counter truth; MCity’s baseline term is not camera-normalized and is reported as a logger result only.

Configuration

Results

Because generated reasoning length is small (~ 8 – 13 tokens), the decode term (0.15 TFLOP) is a rounding error on the fixed 63.81 TFLOP baseline, so total analytical FLOP is effectively constant at ≈ 64.00 TFLOP per inference regardless of token cap. Achieved throughput on the clean Waymo token-cap runs is only ≈ 16.30 TFLOP/s (Table 23)—the model is far from

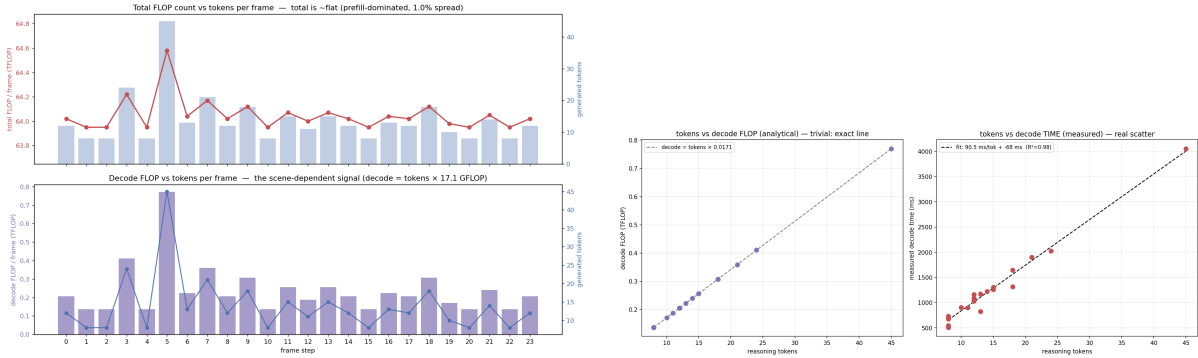
Table 22: Analytical FLOP model configuration.

Parameter	Value
Fixed baseline FLOP	63.81 TFLOP (Waymo 4-camera prompt, $P = 3331$ tokens)
Per-decode-token cost	17.10 GFLOP
Reasoning tokenizer	nvidia/Cosmos-Reason2-8B
Matmul convention	multiply-add counted as 2 FLOPs
Excluded operations	layernorm, softmax, RoPE, bias, residual add
Achieved throughput	total FLOP/ T_i (joined to measured latency)

compute-bound, consistent with the memory-bound decode profile of Section 5. Figures 14a and 14b show the per-frame FLOP vs token relationship and the decode-token contribution.

Table 23: Analytical FLOP and achieved throughput, clean Waymo token-cap runs.

Run	Reason tokens	Decode TFLOP	Total TFLOP	Achieved TFLOP/s
tok16	8.8	0.151	63.96	16.27
tok32	9.5	0.162	63.97	16.43
tok64	9.5	0.162	63.97	16.27



(a) Per-frame analytical FLOP vs reasoning tokens.

(b) Decode-token contribution to total FLOP.

Figure 14: The fixed vision/prefill/expert baseline dominates total FLOP; decode tokens add a negligible slope.

Camera-normalized MCity FLOP

The live logger tags *every* run with the Waymo 4-camera baseline (63.81 TFLOP fixed), which is not camera-correct for MCity’s single camera. Recomputing the fixed FLOP with only the camera-dependent inputs changed ($N_{\text{img}} : 16 \rightarrow 4$, prompt $P : 3331 \rightarrow 1171$, i.e. 720 image tokens + 451 text) yields a much smaller MCity baseline (Table 24). The recomputation is validated by the Waymo column: its 5-step fixed reproduces the logger’s 63.81 TFLOP constant exactly. At the default 5 steps the true MCity fixed FLOP is 22.65 TFLOP, so the Waymo-calibrated logger **over-counts MCity operations by 2.82×**; the gap is dominated by prefill (49.55 TFLOP $\rightarrow$ 16.67 TFLOP) as the prompt shrinks. MCity per-inference FLOP/-TOPS should therefore be reported against this normalized baseline, not the Waymo constant. (Artifact: `scripts/flop_model_camera.py`, `results/flops_camera_normalized/summary.csv`.)

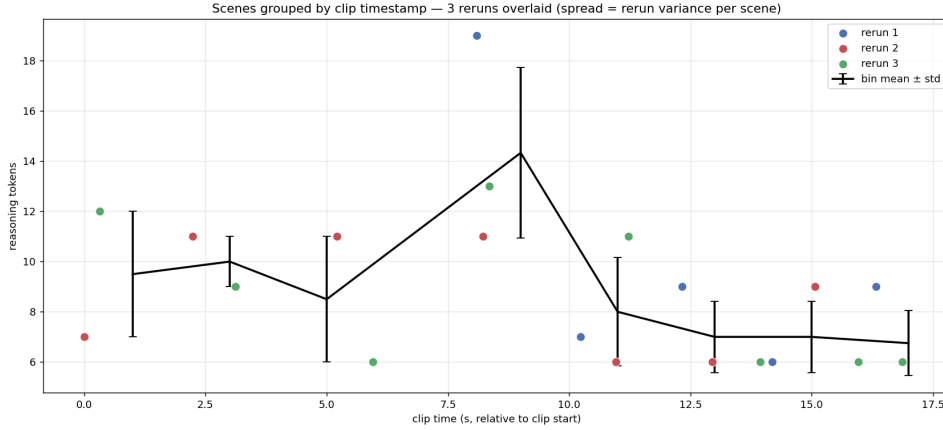


Figure 15: Per-inference analytical FLOP grouped across replay reruns, illustrating run-to-run stability of the operation-count estimate.

Table 24: Camera-normalized analytical fixed FLOP per inference. Only the camera-dependent inputs differ between rows; the Waymo 5-step value reproduces the validated 63.81 TFLOP logger constant.

Dataset	N_{img}	P	Steps	Vision	Prefill	Fixed TFLOP
Waymo (4 cam)	16	3331	2	10.50	49.55	61.55
	16	3331	5	10.50	49.55	63.81
	16	3331	10	10.50	49.55	67.58
MCity (1 cam)	4	1171	2	2.63	16.67	20.64
	4	1171	5	2.63	16.67	22.65
	4	1171	10	2.63	16.67	26.01

10 KV Cache Size

Methodology

KV bytes are summed from the model’s `past_key_values` after VLM generation and before diffusion consumes the cache; a recursive walker handles HuggingFace cache objects, dicts/lists/tuples, and `key_cache/value_cache` attributes while de-duplicating Python object IDs. CUDA peak allocation delta is measured independently by resetting peak stats at inference start and reading peak allocated/reserved after trajectory publication; it captures whole-inference memory pressure (KV + activations + temporaries + diffusion working memory), not KV alone.

Configuration

Table 25: KV / GPU-memory measurement configuration.

Parameter	Waymo	MCity
Cameras / images per inf.	4 / 16	1 / 4
<code>max_generation_length</code>	64	64
<code>num_diffusion_steps</code>	2	2
<code>num_trajectory_samples</code>	1	1
<code>precision_mode</code>	bf16	bf16
CFG / navigation	off (unguided KV = 0)	off
KV measurement point	after VLM generation, before diffusion	

Results

KV footprint scales with prompt length, which scales with camera count: Waymo’s ~ 3084 -token prompt yields 433.60 MiB mean KV vs MCity’s 845-token prompt at 118.90 MiB (Table 26). Yet KV is a small fraction of whole-inference memory pressure—1.96% of the Waymo CUDA peak and 0.55% of MCity’s—so the CUDA peak allocation delta (972.50 MiB Waymo, 268.20 MiB MCity) is dominated by activations and diffusion/expert working memory, not the cache. The trajectory-start token was decoded on 100% of Waymo and 94% of MCity inferences. Figures 16a and 16b show the paired KV bars and the KV-vs-tokens scatter.

Table 26: KV cache and CUDA memory (baseline reruns, mean).

Metric	Waymo	MCity
Prompt KV sequence length	3083.7	845.2
Total KV (MiB)	433.6 (max 437.1)	118.9 (max 125.3)
Model parameters (GiB)	20.64	20.64
CUDA peak allocated (GiB)	21.63	20.92
CUDA peak alloc. delta (MiB)	972.5	268.2
KV % of peak allocated	1.96%	0.55%
Trajectory-start hit rate	1.00	0.94

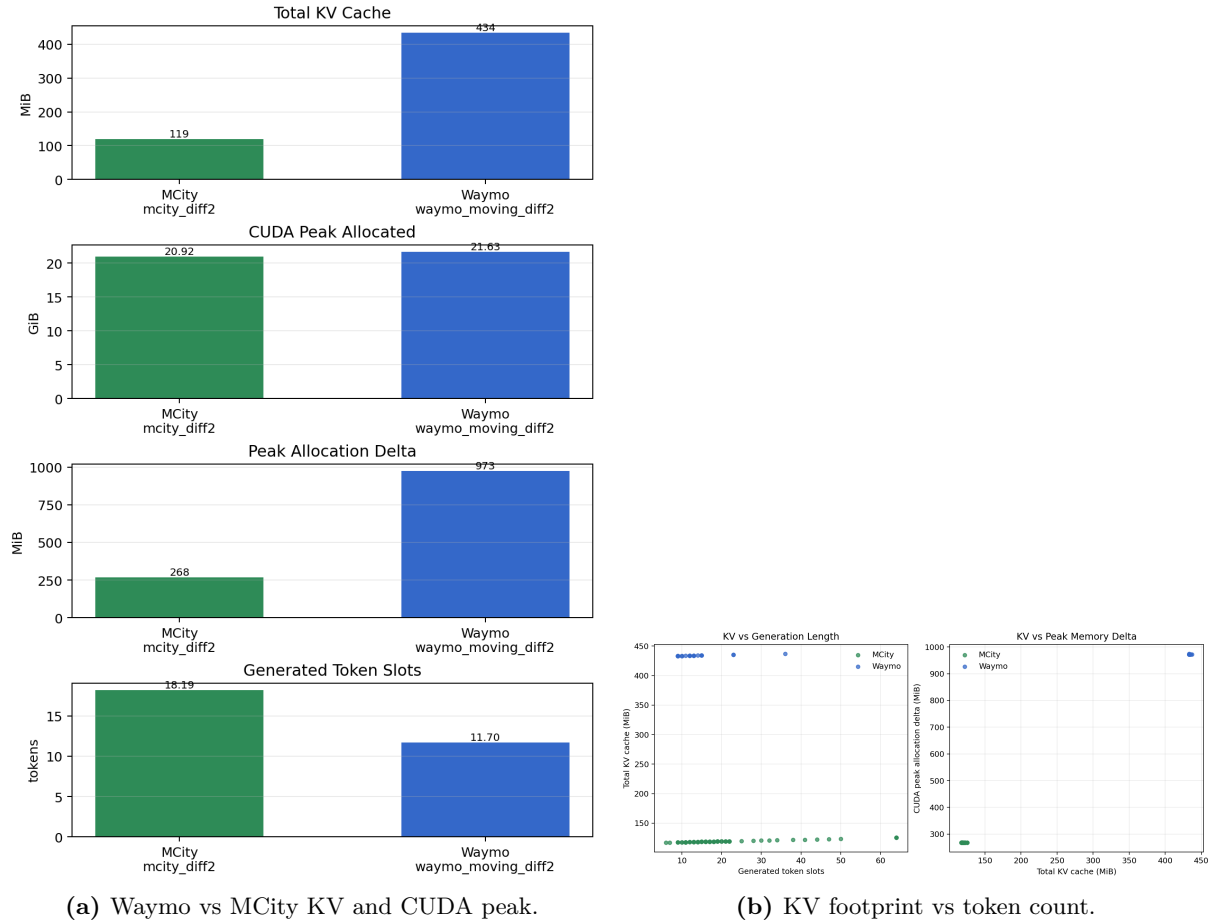


Figure 16: KV cache footprint and its (small) share of total GPU memory pressure.

11 Trajectory-Sample (K) Sweep

Methodology

This sweep varies `num_trajectory_samples` (K) while holding all other knobs at defaults within each dataset. The Autoware Trajectory output publishes sample 0 only, so the reported Waymo ADE/FDE is *sample-0* accuracy; a true best-of-K $\min\text{ADE}_K/\min\text{FDE}_K$ requires logging all returned samples and is tracked as remaining work.

Configuration

Table 27: Trajectory-sample (K) sweep configuration.

Parameter	Waymo	MCity
Swept: <code>num_trajectory_samples</code> (K)	{1, 2, 4}	
<code>max_generation_length</code>	64	64
<code>num_diffusion_steps</code>	5	5
<code>precision_mode</code>	bf16	bf16
Cameras / images per inf.	4 / 16	1 / 4
Replay rate	0.1×	0.5×
Published sample	sample 0 (Autoware Trajectory)	

Results

K is the single most expensive latency/energy lever measured: Waymo latency rises $3010 \rightarrow 4662 \rightarrow 8474$ ms and GPU energy $107 \rightarrow 190 \rightarrow 376$ J as K goes $1 \rightarrow 2 \rightarrow 4$ (Table 28), i.e. close to linear in K because each sample re-runs the diffusion expert. Sample-0 ADE actually *worsens* with K ($0.103 \rightarrow 0.181$ m), which is expected: adding samples does not improve the published sample 0, and any best-of-K benefit is invisible until all samples are scored. Figure 17 shows the full sweep.

Table 28: Trajectory-sample sweep (mean). Waymo accuracy is sample-0 only.

Dataset	K	<i>n</i>	Latency (ms)	GPU E (J)	ADE ₀ (m)	FDE ₀ (m)
Waymo	1	53	3009.6	106.6	0.103	0.184
	2	35	4661.9	190.4	0.138	0.181
	4	21	8473.8	375.7	0.181	0.259
MCity	1	185	2159.5	51.5	N/A	N/A
	2	55	3261.9	85.6	N/A	N/A
	4	89	4903.1	148.2	N/A	N/A

Best-of-K accuracy

The sample-0 accuracy above answers “is the *published* path better with more samples?” (no). The more useful question is “does the model’s sample set *contain* a better path?”—i.e. $\min\text{ADE}_K/\min\text{FDE}_K$. That was previously unanswerable because only sample 0 is published. We added an opt-in all-samples logger to the node (`all_samples_csv`), re-ran Waymo K= 1/2/4 logging every returned sample, and scored each sample against replayed odometry with the same alignment as Section 8 (`scripts/evaluate_best_of_k.py`). Table 29 and Figure 18 show the result: **the published sample 0 is far from the best sample**. Taking the best of K cuts ADE by 41% at K= 2 (0.15 m→0.09 m) and 48% at K= 4 (0.20 m→0.10 m), with similar

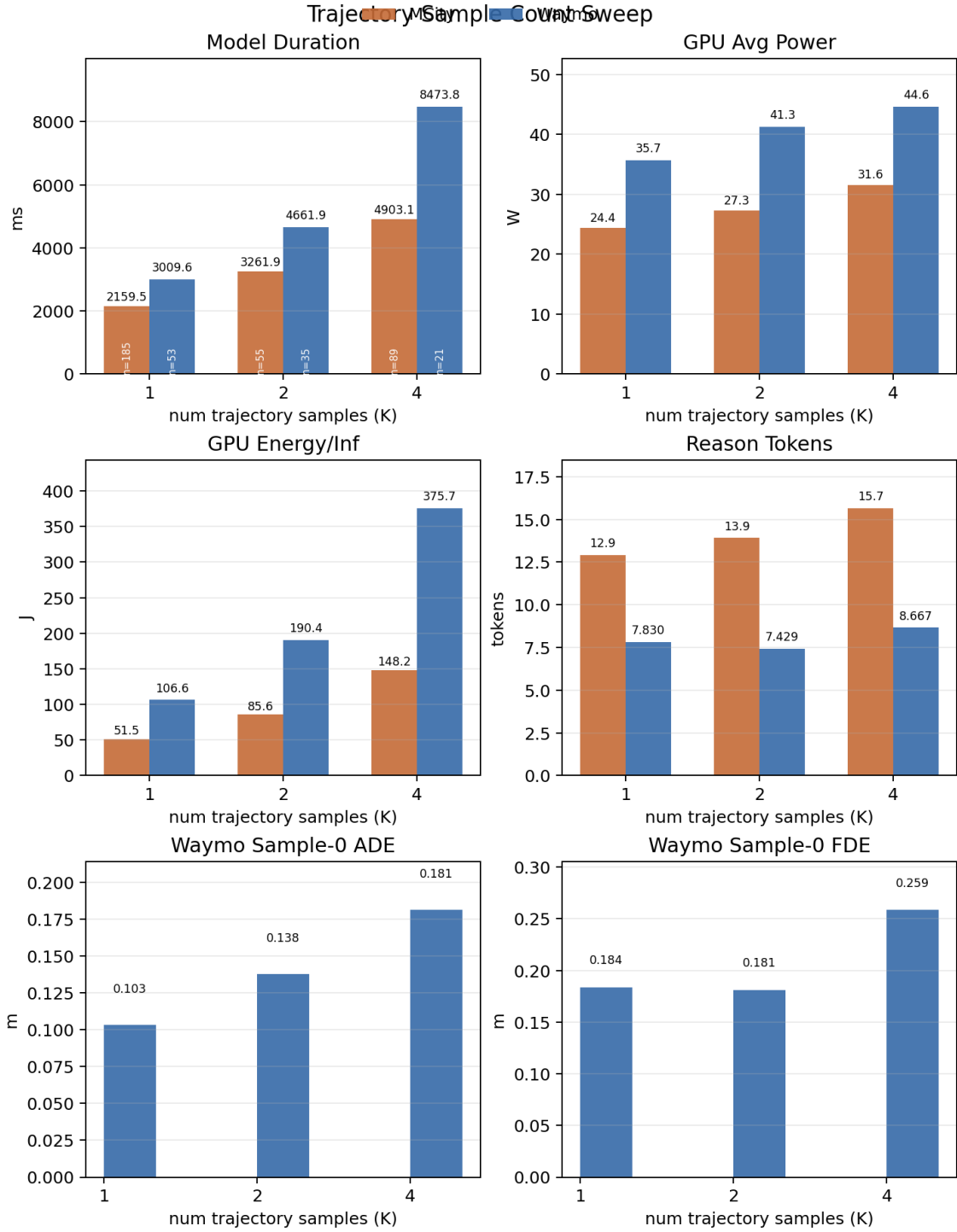


Figure 17: Default-parameter K= 1/2/4 sweep: latency, GPU power/energy, tokens, and Waymo sample-0 ADE/FDE.

37–49% FDE reductions. So the extra samples are not wasted work—their diversity holds materially more accurate trajectories—but capturing that gain requires either publishing/selecting among samples or a learned ranker, since sample 0 alone even degrades slightly as K grows. (The per- K prediction counts differ—53/35/21 over the segment—and $K=4$ ’s $n=21$ is small, so read the within-run sample-0-vs-best gap, which is robust, rather than cross- K minADE.)

Table 29: Best-of- K vs published sample 0 (Waymo, replayed-odometry accuracy).

K	n	ADE_0 (m)	minADE_K (m)	FDE_0 (m)	minFDE_K (m)
1	53	0.115	0.115	0.234	0.234
2	35	0.145	0.085	0.191	0.120
4	21	0.199	0.104	0.310	0.159

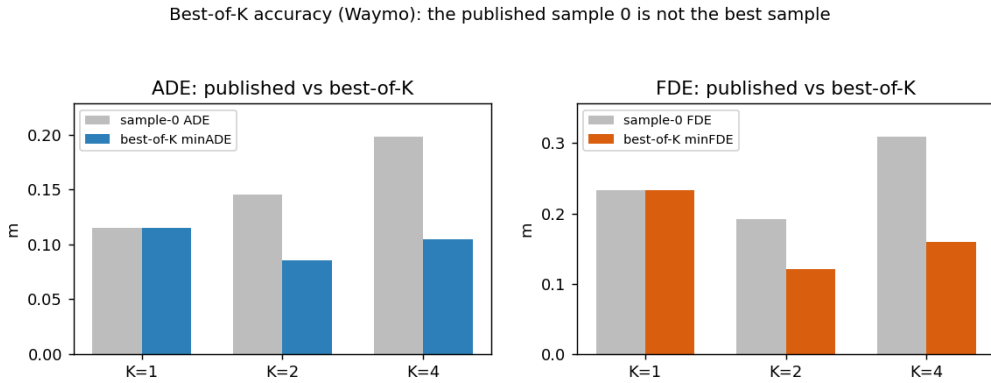


Figure 18: Best-of- K accuracy: the published sample 0 (grey) vs the best of K samples (color). The model’s sample diversity contains substantially more accurate trajectories than the one it publishes.

12 Camera-Count Ablation

Methodology

The Waymo-vs-MCITY comparison of Section 14 confounds camera count with dataset, scene, topic, and route. To isolate the camera-count effect we ran a *controlled* ablation: the same Waymo segment replayed three times with the node consuming 1, 2, or 4 cameras—achieved by handing the node a subset of `camera_topics/camera_indices` (front; front + front-right; all four) while every other parameter and the replayed data are held fixed. The republisher still publishes all four compressed streams; only the node’s subscription set changes. The model accepts a variable camera set without error, so both the cost curve and the accuracy curve are measurable.

Configuration

Results

This is the cleanest cost/accuracy trade in the study (Table 31, Figure 19): every cost metric scales monotonically with camera count, and so does accuracy—in the opposite direction. Going from 1 to 4 cameras **cuts ego ADE** $3.6\times$ ($0.33\text{ m}\rightarrow 0.09\text{ m}$) and **FDE** $6\times$ ($0.91\text{ m}\rightarrow 0.15\text{ m}$), but costs $2.0\times$ latency, $2.9\times$ GPU energy, and $3.7\times$ KV cache (the prompt grows $839\rightarrow 3083$ tokens). The 4-camera point reproduces the canonical baseline (KV 433.50 MiB vs 433.60 MiB; ADE 0.09 m vs the baseline range), which validates the ablation harness. Two points worth flagging:

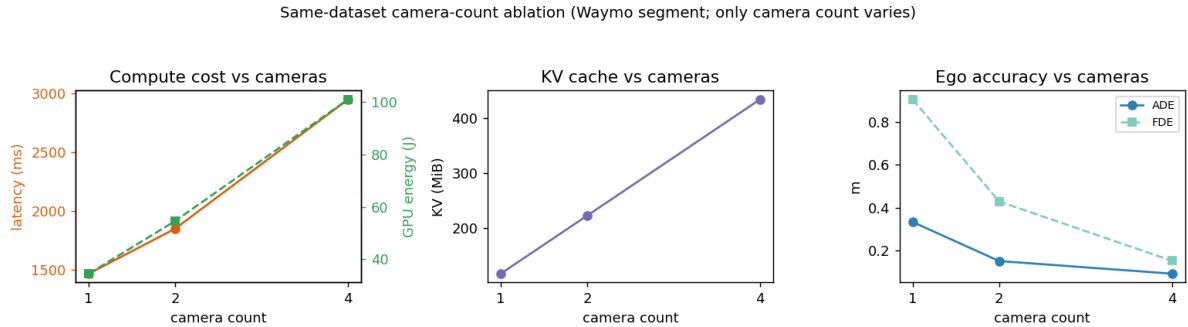
Table 30: Camera-count ablation configuration (same Waymo segment).

Parameter	Value
Swept: camera count	1 (front) / 2 (front + front-right) / 4 (all)
camera_indices	[1] / [1, 2] / [0, 1, 2, 6]
max_generation_length	64
num_diffusion_steps	5
num_trajectory_samples	1
precision_mode	bf16
Replay rate	0.1×

- The single-camera model still plans a usable trajectory (sub-metre ADE) but its FDE nearly reaches 1.00 m—the missing side/telephoto views cost long-horizon accuracy, as expected.
- The live FLOP logger reports ≈ 64.00 TFLOP for all three rows because it uses the fixed Waymo 4-camera constant; the true camera-normalized operation count scales with camera count (Section 9: 20.60 TFLOP at 1 camera vs 63.80 TFLOP at 4), so the real compute saving of dropping cameras is larger than the logged column suggests.

Table 31: Same-dataset camera-count ablation (mean). Only the camera count varies.

Cameras	n	Lat. (ms)	GPU E (J)	KV (MiB)	Prompt tok	ADE (m)	FDE (m)
1	88	1468.9	34.5	117.9	839	0.333	0.905
2	79	1852.0	54.6	223.0	1586	0.151	0.429
4	53	2951.9	101.0	433.5	3083	0.092	0.152

**Figure 19:** Camera-count ablation on a single Waymo segment: compute cost and KV rise with camera count while ego ADE/FDE fall. This isolates the camera-count effect that the cross-dataset comparison could not.

13 Token-Limit Behavior

Methodology

Token-limit behavior records, per run, the actual generated reasoning-token count, whether the `max_generation_length` cap was reached, whether any inference produced zero reasoning tokens, whether the trajectory-start token was decoded, and whether the trajectory was truncated before a valid decode. Cap hits are inferred as `reason_tokens` $\geq$ `max_generation_length`; the trajectory-start hit rate is taken from the KV instrumentation reruns.

Configuration

Table 32: Token-limit-behavior data sources.

Parameter	Value
Runs analyzed	Waymo baseline (<code>max_generation_length=64</code>) + MCity baseline
Cap-hit rule	<code>reason_tokens ≥ max_generation_length</code>
Traj-start hit rate	from KV reruns (generated-token slots)
Truncation markers	token IDs not persisted (inferred, not explicit)

Results

At the default cap of 64 the model never comes close to truncation (Table 33): the maximum observed reasoning length is 20 tokens on Waymo and 47 on MCity, so the max-token hit rate is 0% on both datasets and every inference decoded a complete 64-point trajectory (zero incomplete-trajectory rows). The trajectory-start token was decoded on 100% of Waymo and 94% of MCity inferences. MCity occasionally emits a zero-reasoning-token inference (8 of 137, $\sim 6\%$)—the model still produced a valid trajectory in those cases—whereas Waymo never did. This confirms the finding of Section 4.2: the reasoning-token cap is not an active constraint at the default operating point.

Table 33: Token-limit behavior at the baseline operating point (cap = 64).

Quantity	Waymo	MCity
Inferences	46	137
Reasoning tokens (mean / max)	7.85 / 20	13.33 / 47
Max-token hits (rate)	0 (0%)	0 (0%)
Zero-reasoning-token rows	0	8
Incomplete-trajectory rows	0	0
Trajectory-start hit rate	1.00	0.94

14 Camera Stream Validation and Freshness

Methodology

Camera delivery is measured two ways because `inference_metrics.csv` records only *configured* camera counts, not real receipts. `validate_camera_streams.py` subscribes to the compressed topics the node consumes (plus optional raw topics and `/alpamayo/predicted_trajectory`) and, on shutdown, writes per-topic message counts, measured Hz, dropped-gap counts (inter-frame gap $> 1.5 \times$ median dt), duplicate-stamp counts, cross-camera skew, and trajectory count. Separately, `camera_receipts.csv` yields the two freshness quantities of Section 4.1. The Waymo-vs-MCity comparison is confounded by camera count, dataset, topic, rate, and route, so it is *not* a controlled camera-count ablation (that is listed as remaining work).

Configuration

Results

Under the clean baseline every camera delivers its full frame count at the expected 10.00 Hz with zero drops, zero duplicates, and zero cross-camera skew, and the node publishes a trajectory

Table 34: Camera-validation and freshness configuration.

Parameter	Value
Subscribed topics	compressed camera topics + <code>/alpamayo/predicted_trajectory</code>
Drop threshold	inter-frame gap $> 1.5 \times$ median dt
Reported metrics	msg count, Hz, drops, duplicate stamps, cross-camera skew
Freshness metrics	<code>odom_minus_camera_stamp</code> , <code>camera_staleness</code>
QoS probe matrix	reliable vs. best-effort, node vs. no-node

for every completed inference (Table 35). Heavier clean token-cap sweeps exposed intermittent BEST_EFFORT front-camera loss (e.g. 199/187/172/199), which reliable-QoS / no-node probes isolated to transport backpressure rather than model latency. This motivates the standing recommendation to use reliable QoS for characterization captures where transport fidelity matters.

Table 35: Camera-stream validation at the baseline operating point.

Quantity	Waymo	MCity
Cameras	4	1
Messages / camera	199 (all four equal)	2649
Measured rate (Hz)	10.0	10.0
Dropped-gap count	0	0
Duplicate-stamp count	0	0
Cross-camera stamp skew	0 ms	n/a (single camera)
Predicted trajectories	43	137

15 Navigation-Text Input

Methodology

Final navigation-text experiments use the *stamped* input path, not static parameter overrides. The harness starts `run_nav_text_publisher.sh` when `NAV_TEXT/NAV_TEXT_SCHEDULE` is set; it publishes `autoware_internal_debug_msgs/StringStamped` on `/alpamayo/nav_input` aligned to `/clock`. The node selects the latest nav message whose stamp is not newer than the odometry/input stamp for that inference, and records the effective `nav_text` on each completed inference row, so a valid nav run must show the intended text on completed rows.

Configuration

Table 36: Navigation-text input configuration.

Parameter	Value
Input topic	<code>/alpamayo/nav_input</code>
Message type	<code>autoware_internal_debug_msgs/StringStamped</code>
Publish rate	10.00 Hz (<code>NAV_TEXT_HZ</code>)
Stamp alignment	aligned to <code>/clock</code>
Selection rule	latest nav stamp not newer than the inference input stamp
Proof text	“stop, red light” (stamped)
Method	stamped input (static parameter override deprecated)

Results

We ran a controlled same-settings A/B/C: three Waymo replays identical in every parameter ($\text{MAX_GEN_LEN}=64$, $\text{NUM_DIFF_STEPS}=5$, $\kappa=1$, bf16, rate $0.1\times$) differing only in the stamped nav text—*A*: “drive straight through the intersection”, *B*: “turn right at the intersection”, *C*: “stop, red light ahead”. Each run recorded exactly its intended nav text on every inference (confirming the stamped path end-to-end), and all three went through the CFG-nav dual-pass model call. Two findings emerge (Table 37, Figure 20):

1. **Nav text barely moves the trajectory on this segment.** The pairwise trajectory divergence between prompts is tiny: mean endpoint divergence is 1.40 cm (straight vs right), 3.30 cm (straight vs stop), and 18.70 cm (right vs stop), against ~ 40.00 m-scale paths. Ego ADE stays sub-metre and similar across prompts (0.08 m to 0.11 m). On a moving segment where the road geometry strongly constrains the path, contradictory nav commands are largely overridden by the scene—an important caveat for anyone expecting nav text to steer the planner.
2. **CFG-nav is not free.** The guided+unguided dual pass makes these runs ~ 4.80 s to 5.90 s per inference versus ~ 3.00 s for the non-nav $\kappa=1$ baseline (Section 11)—a $\sim 1.6\times$ latency and $\sim 1.7\times$ GPU-energy cost for conditioning that changes the output only marginally here.

Table 37: Stamped-nav A/B/C (Waymo, same settings; only nav text differs). Divergence is the mean trajectory / endpoint distance between prompt pairs at matching prediction stamps.

Prompt	n	Latency (ms)	GPU E (J)	ADE (m)	FDE (m)
A: straight	29	5897.2	193.4	0.104	0.151
B: right	33	5074.1	185.4	0.106	0.161
C: stop	34	4847.0	183.9	0.082	0.139
<i>Pairwise trajectory / endpoint divergence (m):</i>					
A vs B: 0.005 / 0.014		A vs C: 0.016 / 0.033		B vs C: 0.052 / 0.187	

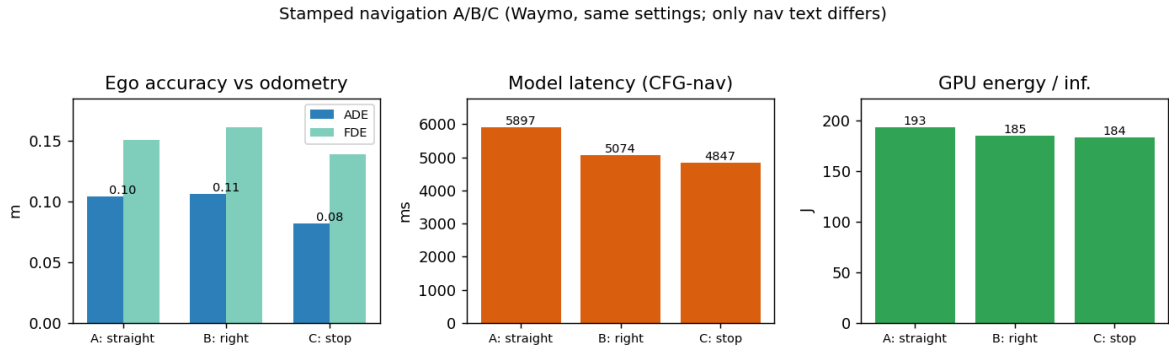


Figure 20: Stamped nav A/B/C: ego ADE/FDE, model latency, and GPU energy. Accuracy is similar across prompts; the elevated latency/energy is the CFG-nav dual-pass cost.

16 Status Audit and Remaining Work

Table 38 is the characterization status, originally audited 2026-08-18 and refreshed 2026-09-02 (✓ done, ◐ partial, ✗ not found). The 2026-09-02 refresh closed five items that were open at the first audit: the MCity camera-normalized FLOP recalculation (Section 9), the dataset-matched

MCity token-cap sweep (Section 4.2), the stamped-nav A/B/C (Section 15), the same-dataset camera-count ablation (Section 12), and best-of-K $\text{minADE}_K/\text{minFDE}_K$ (Section 11). The remaining open items in Table 39 follow the dataset rule: every non-GT characterization should exist for both Waymo and MCity, and GT-dependent eval stays Waymo-only until a trusted MCity future-trajectory target is added.

Table 38: Characterization status audit (2026-08-18, refreshed 2026-09-02).

Characterization item	Status
Latency: stage, frame, dataset, token-cap, diffusion	partial
Nsight profiling (CUDA API, kernels, stage bottlenecks)	done
Operation count / achieved TFLOP/s (camera-normalized)	done
Camera-count analysis (controlled same-dataset ablation)	done
Energy & power: avg/peak/per-inference/datasets/sweeps	partial
Accuracy: ADE/FDE + token/diffusion/camera effects + best-of-K	done
Dataset-matched diffusion-step sweep	done
Trajectory-sample (K) replay + best-of-K $\text{minADE}/\text{minFDE}$	done
Precision BF16/FP16 default-parameter replay	done
Precision FP8/INT8 end-to-end replay	not done
KV-cache and GPU memory	done
Sustained throughput / FPS / drops / input age	done
Fixed input configuration documentation	done
Token-limit behavior	done
Stamped navigation-text A/B/C	done

Table 39: Prioritized remaining work.

#	Item	Waymo	MCity
1	MCity camera-normalized FLOP/TOPS recalculation	done	done
2	Dataset-matched token-cap sweep	done	done
3	Stamped-nav A/B/C at identical settings	done	not done
4	Same-dataset camera-count ablation	done	n/a
5	All-samples $\text{minADE}_K/\text{minFDE}_K$ for K-sweep	done	n/a
<i>Still open:</i>			
6	MCity accuracy target decision (GT/odometry)	n/a	not done
7	FP8/INT8 end-to-end precision runtime + replay	not done	not done
8	Energy per stage (NVTX-aligned integration)	not done	not done
9	Best-of-K sample selection / ranker in the node	not done	n/a

17 Conclusion

This characterization establishes a reproducible, artifact-backed picture of Alpamayo 1.5 as a deployed real-time planner on Jetson AGX Thor. Four findings stand out for anyone tuning the system for deployment:

1. **The system is memory-bound in autoregressive decode.** Reason decode is 71% of GPU time and the model achieves only ≈ 16 TFLOP/s, so latency tracks decode length, not raw FLOPs.

2. **Trajectory-sample count (K) and precision are the expensive levers.** K scales latency/energy roughly linearly; FP16 is both slower and higher-energy than the BF16 default on this stack, with no accuracy benefit.
3. **The reasoning-token cap is nearly inert** on these segments because generated length never approaches the cap—diffusion steps and K are the real cost knobs.
4. **Accuracy is stable and safe at the sanity level.** Ego ADE/FDE stays sub-metre across all runnable Waymo configurations and the GT actor check finds zero collisions, though these are sanity/clearance checks, not the official planning benchmark.
5. **Camera count is the strongest accuracy lever—and it is expensive.** The controlled same-dataset ablation shows 1→4 cameras cuts ADE 3.6× and FDE 6× while costing 2.9× GPU energy and 3.7× KV; the camera set, not the reasoning knobs, dominates accuracy.
6. **The published trajectory is not the model’s best one.** Best-of-K recovers 40–50% lower ADE/FDE than the published sample 0, so a sample selector or ranker is a high-value, no-retrain accuracy win. Conversely, stamped nav text barely steers the planner on a scene-constrained segment (≤ 0.19 m endpoint change) while the CFG dual pass costs $\sim 1.6\times$ latency.

The most valuable open work is now an end-to-end FP8/INT8 runtime (the only precision path that could plausibly beat BF16), stage-level energy via NVTX-aligned power integration, a best-of-K sample selector/ranker in the node to capture the accuracy the model already produces, and a trusted MCity future-trajectory target so that MCity can move from N/A to real accuracy reporting.